

Proof of concept: wind-adaptive energy optimisation in a head-wind

Step-by-step guide • the autonomous layer • flight dynamics, logs and criteria • the simulation process • how the lab applies optimisation

Baseline mission energy 64.5 Wh fixed 17 m/s	Optimised mission energy 58.8 Wh $V = 15.5 + 0.64 \cdot H$	Energy saving 8.8 % same model, wind, seed	Criteria passed 5 / 6 fixed before the run
---	--	---	---

uavlab 0.1.0 • JSBSim 1.3.1 • Python 3.11.15 • 2026-09-24
Reproduce: `python poc/run_poc.py` (results in `poc_results/`)

Contents

Summary	3
1 The proof of concept at a glance	4
1.1 Hypothesis	4
1.2 Experimental design	4
1.3 Success criteria (fixed before the experiment)	4
1.4 Why this design is valid and fast	5
2 The autonomous layer: its logic and how it is simulated	6
2.1 Where autonomy sits	6
2.2 Flight phases (mode machine)	6
2.3 Decisions	7
2.4 What the autonomy sees	8
2.5 The policy slot: where the optimised controller acts	8
2.6 How the autonomy is simulated	9
3 Flight-dynamics model, logs and criteria	10
3.1 Flight-dynamics model	10
3.2 Energy accounting	10
3.3 What is logged	11
3.4 Criteria	12
4 The simulation process	13
4.1 One run from start to finish	13
4.2 One time step	13
4.3 Determinism and seeds	13
5 How the lab applies optimisation	14
5.1 Four routes into an optimising controller	14
5.2 The optimisation problem of this PoC	14
5.3 The cross-entropy method, step by step	15
5.4 What the optimiser found	16
5.5 The physics behind the result: speed-to-fly	16
5.6 From this PoC to the adaptive optimising controller	17
6 Step-by-step guide to run the proof of concept	18
Route A: one command (about 21 min on 2 cores, less on more cores)	18
Route B: in the graphical interface (no code)	18
Troubleshooting	19
7 Results	20
7.1 Comparison in the PoC condition	20
7.2 Where the saving comes from: leg by leg	21
7.3 Robustness in unseen turbulence	22
7.4 Criteria	23
7.5 Interpretation and limits	23
Appendix A: Configuration of the experiment	25
Appendix B: Files and commands	26

Summary

Question. Can an optimisation method, applied to the autonomy layer of the UAV, reduce the battery energy needed to fly a mission in a head-wind? The comparison must be made on the same aircraft model, in the same wind and with the same random seed.

Experiment. The Rascal 110 electric UAV flies the same out-and-back mission twice in a steady 6 m/s westerly wind, which is 8.3 m/s at the 100 m cruise height. The mission has two up-wind legs and two down-wind legs.

- **Baseline:** the aircraft flies the mission's fixed cruise speed of 17 m/s on every leg.
- **Optimised:** the autonomy layer flies a wind-adaptive speed law, $V = v_0 + k_{head} H$, where H is the head-wind the aircraft estimates in flight. The lab's **cross-entropy optimiser** (CEM) chooses the two parameters v_0 and k_{head} . It flew 40 complete simulated flights, from take-off to full stop, to find them.

Result. The optimiser found $v_0 = 15.50$ m/s and $k_{head} = 0.639$. The aircraft now flies 20.8 m/s into the wind and 12.0 m/s with the wind.

- **Mission-leg energy:** it fell from **64.49 Wh** to **58.82 Wh (8.8 % less)**.
- **Whole flight** (take-off to stop): 7.9 % less energy.
- **Pre-stated criteria: 5 of 6 passed.** The saving holds in three turbulence seeds the optimiser never saw (C5). The physics cross-check C4 missed its 1.5 m/s tolerance by 0.03 m/s: the optimised up-wind speed is 20.8 m/s, and the fitted speed-to-fly formula gives 19.3 m/s. The post-hoc diagnosis in Section 5.5 traces this to the power-curve fit: with the measured map points the physics optimum is 20.4 m/s.

Conclusion. The proof of concept succeeds on its main question, with one pre-stated criterion missed. The model, route, wind and seed were the same for both flights. With them, the optimised wind-adaptive speed law needed 8.8 % less mission-leg energy than the fixed cruise speed. The optimised flight completed the same mission safely, and the saving held (9.4–9.6 %) in three turbulence realisations the optimiser never saw. Criterion C4 failed by 0.03 m/s: the up-wind speed is 20.8 m/s, against 19.3 m/s from the fitted speed-to-fly formula. The post-hoc diagnosis in Section 5.5 traces the gap to the 3-parameter power-curve fit, not to the simulation or the optimiser. The lab's pipeline is a valid basis for the adaptive energy-optimising controller: full-flight simulation, logs, pre-stated criteria, and a derivative-free optimiser acting at the autonomy layer.

How to reproduce. From the repository folder, run `python poc/run_poc.py`, or open `poc/run_poc.py` in an IDE and press Run. It takes about 21 minutes on the 2-core machine used for this report, and less on a 4–8 core PC. Section 6 gives the step-by-step guide, including a route through the graphical interface that needs no code.

1 The proof of concept at a glance

1.1 Hypothesis

In a head-wind the ground speed drops, so each kilometre takes longer and costs more energy. Classical flight mechanics predicts that the energy per kilometre is minimised by flying **faster into the wind and slower with the wind** (the speed-to-fly principle, Section 5.5). The hypothesis is:

A wind-adaptive airspeed law whose parameters are found by the lab's optimiser, using only simulated flights, uses measurably less battery energy than the fixed cruise speed. This holds on the same route, in the same wind, with the same aircraft and the same random seed.

1.2 Experimental design

Only one thing differs between the two flights: the rule that sets the airspeed. Everything else is identical and fixed by configuration.

Item	Baseline	Optimised
Aircraft, powertrain, controllers	Rascal 110e, defaults	identical
Mission	out_and_back : legs to W_END, E_END, W_END2, E_END2 at 100 m	identical
Wind	steady 6 m/s from 270° at 10 m, power-law shear ($\alpha = 1/7$)	identical
Random seed (sensor noise)	1	1
Airspeed rule (autonomy policy)	fixed : 17 m/s	linear_wind : $V = v_0 + k_{head}H$, clamped to 12–26 m/s
How the rule was chosen	mission default	cross-entropy optimisation, 40 flights

The steady wind makes the comparison deterministic: running the same flight again gives the same numbers (V&V check V12). So a single paired comparison is valid, and no averaging is needed. To make sure the result is not an artefact of that idealised wind, step 4 repeats both designs in light Dryden turbulence with three seeds the optimiser never saw.

1.3 Success criteria (fixed before the experiment)

ID	Criterion (fixed before the experiment)
C1	Both flights complete the mission and land on the runway (status LANDED, return reason mission_complete)
C2	Mechanical energy balance closes in both flights: $ \text{residual} < 0.1\%$ of gross work
C3	The optimised speed law needs at least 2 % less mission-leg battery energy than the baseline
C4	Optimised up-wind and down-wind airspeeds agree with the independent speed-to-fly prediction within 1.5 m/s
C5	The saving holds in each of 3 unseen light-turbulence seeds (paired comparison, complete missions)
C6	No safety regression: touchdown sink < 2.5 m/s and no energy-reserve decision (D4/D5) in the optimised flight

1.4 Why this design is valid and fast

- **Controlled.** One factor changes, and every other input is identical, including the random numbers (common random numbers).
- **Complete flights.** Each evaluation is a full take-off-to-full-stop flight with all autonomy decisions active. The saving is therefore not a steady-state estimate.
- **Independent checks.** The simulation is checked against physics (C4, using the performance map, which is an independent model). The optimum is checked against unseen conditions (C5). The energy bookkeeping is checked against conservation of energy (C2).
- **Cheap.** 49 flights in total. The optimiser flies each generation in parallel on all CPU cores.

2 The autonomous layer: its logic and how it is simulated

2.1 Where autonomy sits

The lab mirrors a real autopilot stack. The **autonomous layer** decides *what* to do: which flight phase, which waypoint, whether there is enough energy, and which airspeed and height to fly. The **controller layer** decides *how*: which bank angle, pitch angle and throttle achieve it. The control surfaces and the JSBSim flight dynamics then produce the motion. The autonomy never sees the true state, only sensor measurements with realistic errors.

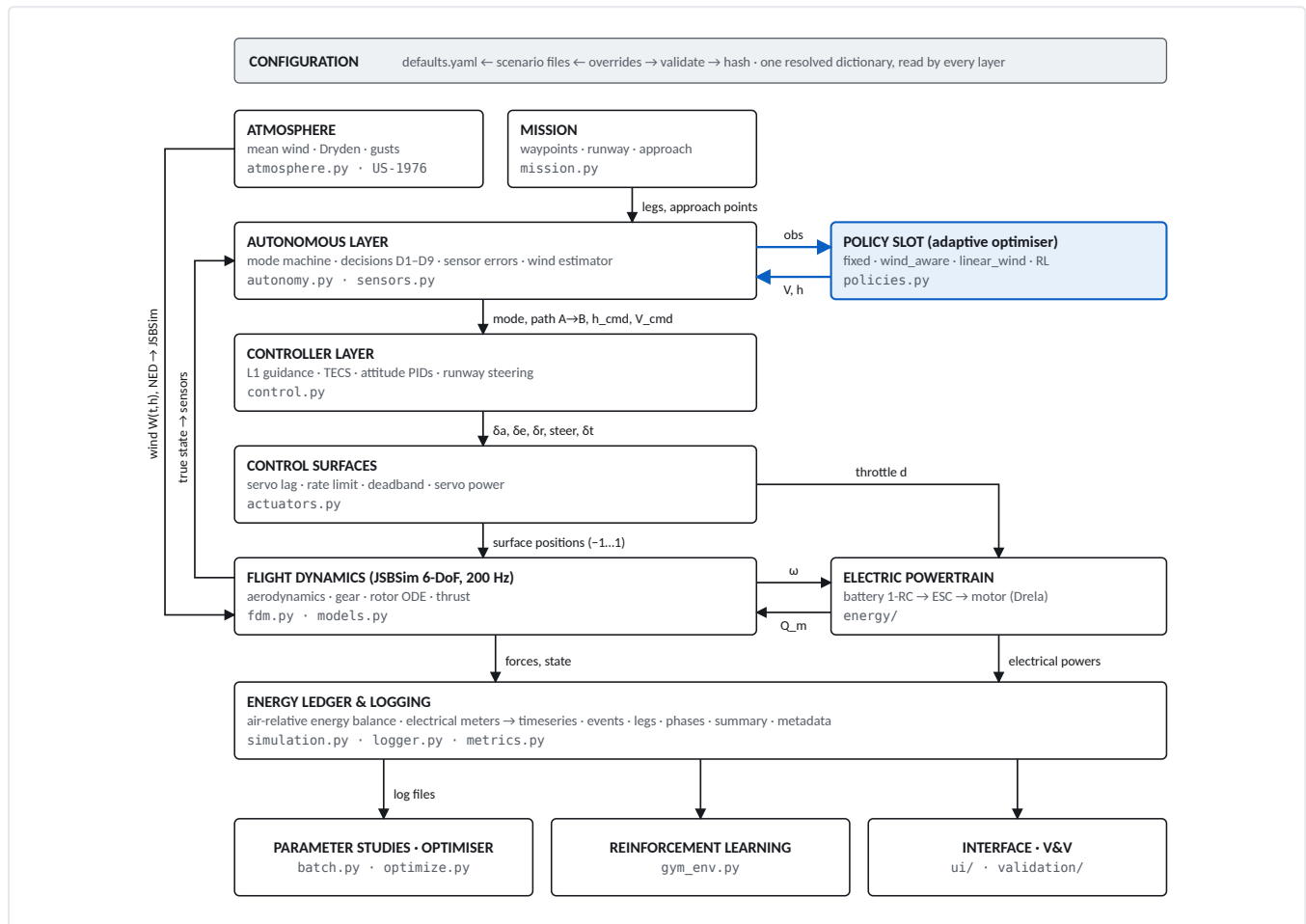


Figure 1. The simulation pipeline. Each layer is one module and one configuration section. Rotor speed and motor torque couple JSBSim and the powertrain every physics step. The **policy slot** (highlighted) is where an adaptive energy-optimising controller plugs in.

2.2 Flight phases (mode machine)

The autonomy is a state machine. Each mode has its own guidance targets, and each transition has an explicit condition that is logged with its reason.

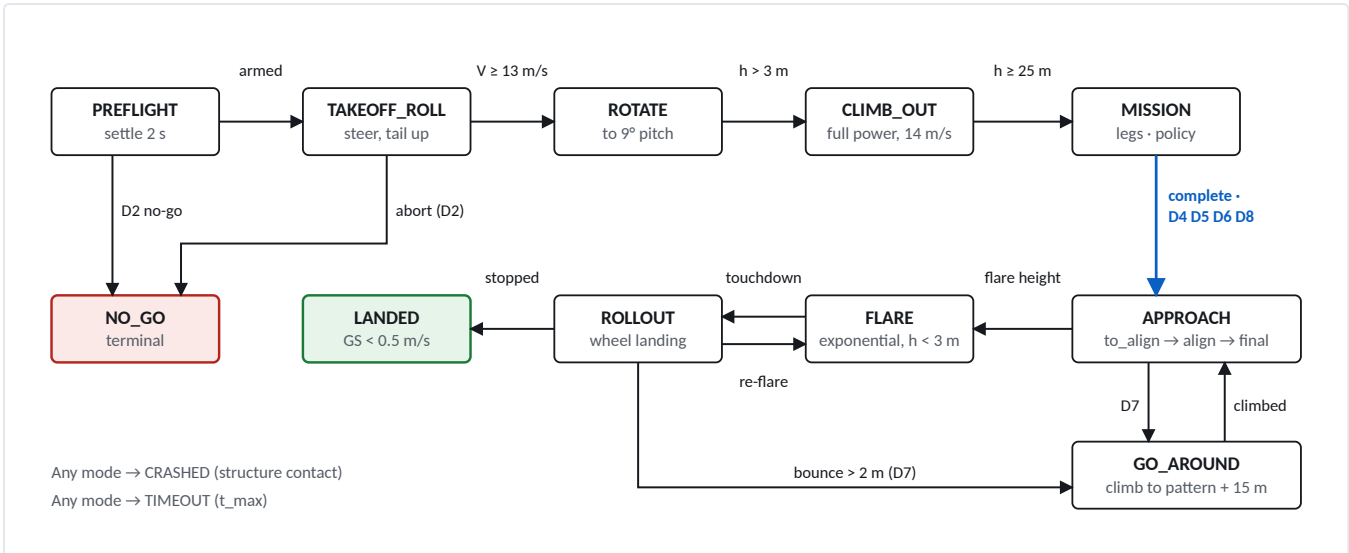


Figure 2. Flight modes and the transitions between them. Energy and safety decisions (D4, D5, D6, D8) end the mission early by switching to APPROACH. Go-around (D7) loops back to the approach. Terminal states are LANDED, NO_GO, CRASHED and TIMEOUT.

Mode	What the autonomy does	Leaves when
PREFLIGHT	settles 2 s on the gear; decides runway and go/no-go (D1, D2)	armed
TAKEOFF_ROLL	full throttle in 1.5 s; steers on the centre line; tail up at 6 m/s	13 m/s → ROTATE (abort after 25 s or at the runway end)
ROTATE	itches up to 9° at 6°/s	3 m above the ground
CLIMB_OUT	full power, speed priority at 14 m/s	25 m → MISSION
MISSION	flies the legs; calls the policy every 20 ms for airspeed and height	last waypoint, or an energy/safety decision
APPROACH	flies to the align point, joins the centre line, follows a 6° glide path	3 m → FLARE
FLARE, ROLLOUT	exponential flare, wheel landing, stops on the runway	ground speed < 0.5 m/s → LANDED

2.3 Decisions

Every decision is a rule with a measurable condition. The decisions that matter in a head-wind are highlighted.

ID	Rule	Action	Role in this PoC
D1	runway heading with the largest head-wind	choose runway	take-off and landing to the west (into wind)
D2	10 m wind > 13 m/s or cross-wind > 6 m/s	refuse flight / abort take-off	6 m/s: go
D3	acceptance radius, passed waypoint or turn-anticipation distance	next leg	sequences the 4 legs
D4	SOC < 25 % (RTL) or < 12 % (land now); cell < 3.45 V	return now	must not trigger
D5	predicted SOC at touchdown for the rest of the route, using the estimated wind, < 15 % for 10 s	early return	energy feasibility in wind; must not trigger

ID	Rule	Action	Role in this PoC
D6	outside the 3 km geofence or above the ceiling	return	-
D7	unstable final approach or bounce > 2 m	go-around / re-flare	-
D8	battery empty	glide landing	-
D9	gust level on final	add up to 2 m/s to the approach speed	smooth air: 0

D5 is itself a small model-based energy prediction. Once per second it evaluates

$$\widehat{\text{SOC}}_{\text{land}} = \text{SOC} - \frac{1}{E_{\text{cap}}} \sum_i \frac{\hat{P} d_i}{\sqrt{V_a^2 - W_{c,i}^2} + W_{a,i}}$$

over the remaining legs i (length d_i), using the measured cruise power $\hat{P}$ and the estimated tail- and cross-wind components W_a , W_c of each leg.

2.4 What the autonomy sees

The autonomy works on **measurements**, not on the truth. Each measurement is the true value plus an error:

- **Navigation values** (airspeed, GPS position and velocity, baro height, attitude, heading) carry slowly varying Gauss-Markov errors, like the output of an EKF: $\sigma = 0.3$ m/s for airspeed, 0.6 m for position, 0.3° for attitude.
- **Gyros and accelerometers** carry white noise.

The **wind estimator** solves the wind triangle from these measurements:

$$\hat{\mathbf{W}} = \mathbf{V}_g - V_a \cos\theta [\cos\psi, \sin\psi]^T$$

It filters the result twice: a fast estimate ($\tau = 5$ s) and a slow mean ($\tau = 60$ s). The slow mean, resolved along the current leg, is the head-wind H that the optimised speed law uses. The aircraft therefore adapts to the wind it *estimates*, as a real UAV would. It is never told the true wind.

2.5 The policy slot: where the optimised controller acts

In MISSION mode, every control step (50 Hz), the autonomy calls the **policy** with an observation: SOC, battery power, airspeed, ground speed, fast and slow wind estimates, head- and cross-wind on the leg, gust level, distance to the waypoint. The policy returns an airspeed set-point, a height set-point or both. The autonomy clamps them to safe limits: airspeed between $1.3 \times$ stall speed and $0.85 \times V_{NE}$, height between 20 m and the geofence ceiling. It then hands them to the controllers, which fly them. The policy cannot touch take-off, landing or the inner loops, so no policy value can crash the aircraft through its low-level control.

Policy	Law	Used here as
fixed	$V = V_{\text{cruise}} = 17$ m/s	baseline
linear_wind	$V = v_0 + k_{\text{head}} H + k_{\text{cross}} W_c + k_{\text{gust}} \sigma_{\text{gust}}$	optimised design ($k_{\text{cross}} = k_{\text{gust}} = 0$)
wind_aware_best_range	$V = \text{argmin}_V P(V)/V_g(V)$ online	physics reference (Section 5.5)
external	set by an outside program	reinforcement learning (Gymnasium)

2.6 How the autonomy is simulated

The autonomy is Python code (`uavlab/autonomy.py`) executed inside the simulation loop every 4th physics step, that is at 50 Hz. It reads the sensor model's output and writes set-points and surface commands. Every mode change and every decision is written to `events.csv` with its time, its reason and the numbers behind it. The whole behaviour is therefore auditable after the flight. Here is the timeline of the baseline flight of this PoC:

Time (min:s)	Kind	Event	Data (selection)
0:00	decision	D1_runway	heading_deg 270.0, reason into_wind, headwind_mps 6.0
0:00	decision	D2_go	wind_mps 6.0
0:02	mode	PREFLIGHT->TAKEOFF_ROLL	reason armed
0:04	mode	TAKEOFF_ROLL->ROTATE	reason rotate_speed, tas 13.01
0:05	mode	ROTATE->CLIMB_OUT	reason airborne, h 3.0, tas 16.12
0:09	mode	CLIMB_OUT->MISSION	reason safe_altitude, h 25.0
4:33	waypoint	reached	name W_END, dist_m 213.1
7:33	waypoint	reached	name E_END, dist_m 444.2
16:18	waypoint	reached	name W_END2, dist_m 210.8
18:43	waypoint	reached	name E_END2, dist_m 39.8
18:43	mode	MISSION->APPROACH	reason mission_complete
19:04	approach	align_reached	
19:50	approach	final	h 44.4, xtrack_m 0.4
22:02	mode	APPROACH->FLARE	reason flare_height, h 2.98, sink_mps 0.78, tas 12.74
22:05	mode	FLARE->ROLLOUT	reason touchdown, sink_mps 0.64, tas 10.83, along_m 30.2, xtrack_m 0.4
22:12	mode	ROLLOUT->LANDED	reason stopped, along_m 52.5, xtrack_m 0.16

3 Flight-dynamics model, logs and criteria

3.1 Flight-dynamics model

The motion is computed by **JSBSim**, an open-source 6-degree-of-freedom flight-dynamics engine used in research and industry. It runs the Rascal 110 model from the ArduPilot SITL (documented changes M1–M10). JSBSim integrates the rigid-body equations with the aerodynamic, propeller, gravity and landing-gear forces at 200 Hz:

$$m \left(\dot{\mathbf{V}}_b + \boldsymbol{\omega} \times \mathbf{V}_b \right) = \mathbf{F}_{aero} + \mathbf{F}_{prop} + \mathbf{F}_{gear} + m \mathbf{g}_b, \quad \mathbf{I} \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{I} \boldsymbol{\omega} = \mathbf{M}$$

The **electric powertrain** is computed by the lab and coupled to JSBSim every step:

- **Propeller:** APC 18x8E data, $T = C_T \rho n^2 D^4$, $P = C_P \rho n^3 D^5$ as functions of advance ratio and RPM.
- **Motor:** Drela model, $E = \omega / K_v$, $Q = (I - I_0) / K_v$.
- **ESC:** averaged, with conduction, switching and quiescent losses.
- **Battery:** 6S LiPo, open-circuit voltage as a function of SOC, series resistance and one RC pair.

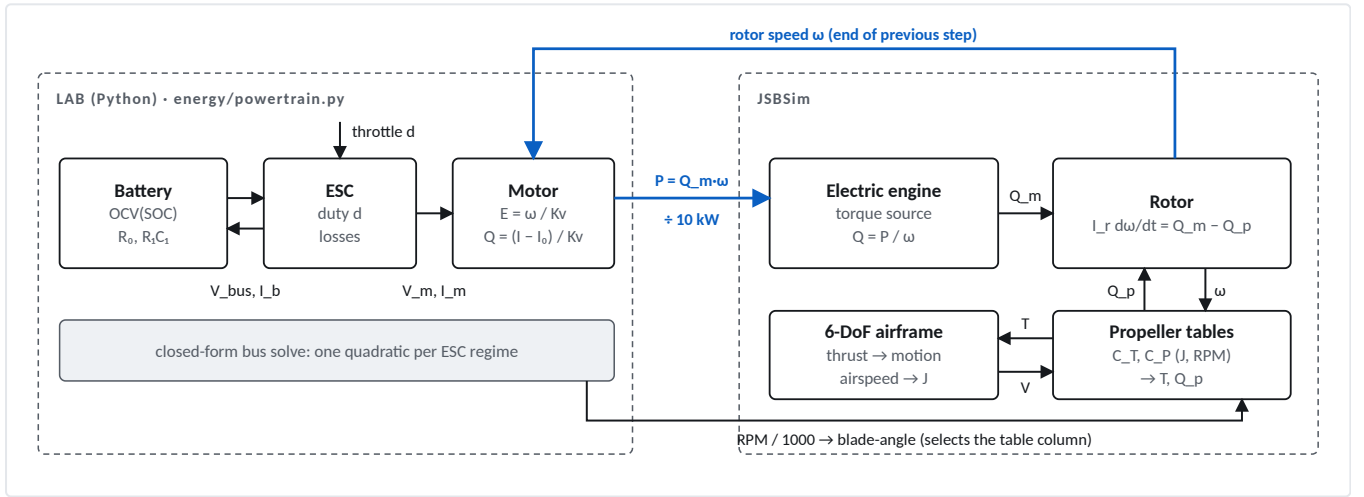


Figure 3. Co-simulation of the electric powertrain with JSBSim, once per 5 ms step. The highlighted arrows are the coupling: motor torque in (encoded as power ÷ 10 kW), rotor speed back out with a one-step lag. The RPM also selects the column of the 2-D propeller tables through the blade-angle input.

The **wind** is a steady westerly. Its speed increases with height following the power law:

$$U(h) = U_{10} \left(\frac{h}{10 \text{ m}} \right)^{1/7} \Rightarrow U(100 \text{ m}) = 6 \times 10^{1/7} = 8.34 \text{ m/s}$$

For the robustness step, MIL-F-8785C Dryden turbulence is added. Its intensity follows the 20 ft wind, which gives light turbulence.

3.2 Energy accounting

Every joule is tracked from the battery chemistry to the air. The cumulative meters are integrated at 200 Hz. The mechanical energy of the aircraft relative to the air, $E_{air} = \frac{1}{2} m V_a^2 + mgh$, must obey

$$\frac{dE_{air}}{dt} = \mathbf{F} \cdot \mathbf{V}_a - mgW_D - m \mathbf{V}_a \cdot \dot{\mathbf{W}}$$

The simulation checks this balance continuously. The remainder is logged as the closure residual, and criterion C2 limits it.

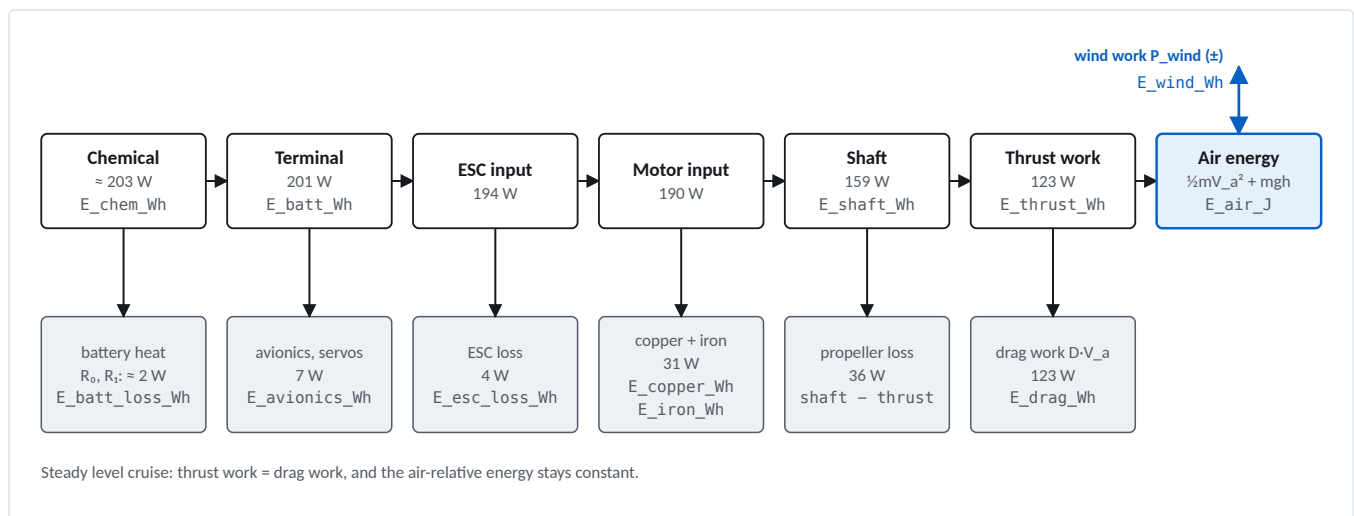


Figure 4. Where the battery energy goes, with typical values in steady cruise at 17 m/s. The loss boxes name the cumulative log columns. The wind term (highlighted) is the only input that does not come from the battery. It can be positive or negative, and it is what wind-exploiting strategies act on.

3.3 What is logged

File	Content	Used in this PoC for
<code>timeseries.csv</code>	about 110 signals at 10 Hz: state, wind, set-points, electrical and mechanical power, cumulative energies	Figures 9 and 11
<code>events.csv</code>	mode changes and decisions with their data	timeline (Section 2.6), criteria C1 and C6
<code>legs.csv</code>	per mission leg: energy, Wh/km, mean airspeed and ground speed, tail-wind, efficiencies	Figure 10, leg table
<code>summary.json</code>	status, energies, breakdown, efficiencies, touchdown, closure residual	results table, criteria
<code>metadata.json</code> , <code>config_resolved.yaml</code> , <code>jsbsim/</code>	versions, seed, configuration and its hash, exact model files	reproducibility

The PoC uses these key quantities:

Quantity	Definition	Log
Mission-leg energy	battery energy used on the four mission legs (optimisation objective)	<code>summary.mission_E_Wh</code>
Total energy	battery energy from take-off roll to full stop	<code>summary.E_batt_Wh</code>
Energy per km	leg energy ÷ ground distance of the leg	<code>legs.Wh_per_km</code>
Head-/tail-wind	wind component along the ground track	<code>tailwind_mps</code> , <code>legs.tailwind_mean_mps</code>
Closure residual	energy balance error ÷ gross work	<code>summary.energy_closure.relative</code>

3.4 Criteria

Two kinds of criteria apply.

Model credibility. The lab's verification and validation suite checks every model against a published reference, an analytic solution or an independent re-implementation. The checks most relevant to this PoC are below; all 19 checks pass in the shipped report.

ID	Check	Criterion	Measured	
V9	Steady flight: full pipeline vs independent trim model	battery power within 3 %, RPM within 1 %, angle of attack within 0.3 deg at every speed; logged lift within 2 % of weight	max errors: P 2.18 %, rpm 0.38 %, alpha 0.030 deg, lift/weight 1.69 %	PASS
V10	Mechanical energy closure in flight	$ dE_{air} - (aero + propulsive + gear + wind \text{ work}) < 0.1 \%$ of gross work for the lab's wind models (0.2 % with JSBSim's turbulence, whose rotational gust terms are not in the closure)	worst 0.04 of limit	PASS
V11	Time-step convergence	total battery energy and flight time change < 1 % when the physics step is halved (5 ms -> 2.5 ms)	energy 0.018 %, time 0.027 %	PASS
V12	Reproducibility	identical seed -> bit-identical time series; different seed -> different turbulence realisation	identical=True, differs=True	PASS
V14	Wind-energy physics on head-wind / tail-wind legs	steady-leg battery energy within 3 % of $E = P(V_a) d / V_g$ (still-air performance map + configured wind profile), and head-wind legs cost > 2x tail-wind legs per km	worst 1.63 %	PASS
D1	Demonstration: wind-aware speed-to-fly policy (online optimiser slot)	informative: same route and wind, fixed 17 m/s vs wind_aware_best_range policy	mission-leg energy 64.49 -> 60.33 Wh (+6.4 % saving)	PASS

PoC evaluation. Criteria C1-C6 (Section 1.3) judge this experiment. The objective is the mission-leg energy, not the total. Take-off, climb, approach and landing are nearly identical in both flights and would dilute the effect. The total is reported as well.

4 The simulation process

4.1 One run from start to finish

1. **Configuration.** `configs/defaults.yaml` is merged with the mission and wind files and the overrides (policy, seed), then validated. The result is one resolved configuration with a hash.
2. **Model files.** The lab writes the exact JSBSim aircraft, engine and propeller files for this run and records their SHA-256.
3. **Set-up.** The lab builds the wind field, mission, powertrain, actuators, sensors, logger and autopilot. The autonomy takes the pre-flight decisions (D1 runway, D2 go/no-go), and the aircraft is placed on the runway.
4. **Time loop.** It runs 5 ms steps until the aircraft stops (LANDED), crashes or reaches the time limit. A 22-minute flight is about 266 000 steps.
5. **Outputs.** The lab writes the logs of Section 3.3 and computes the per-leg and per-phase tables and the summary.

4.2 One time step

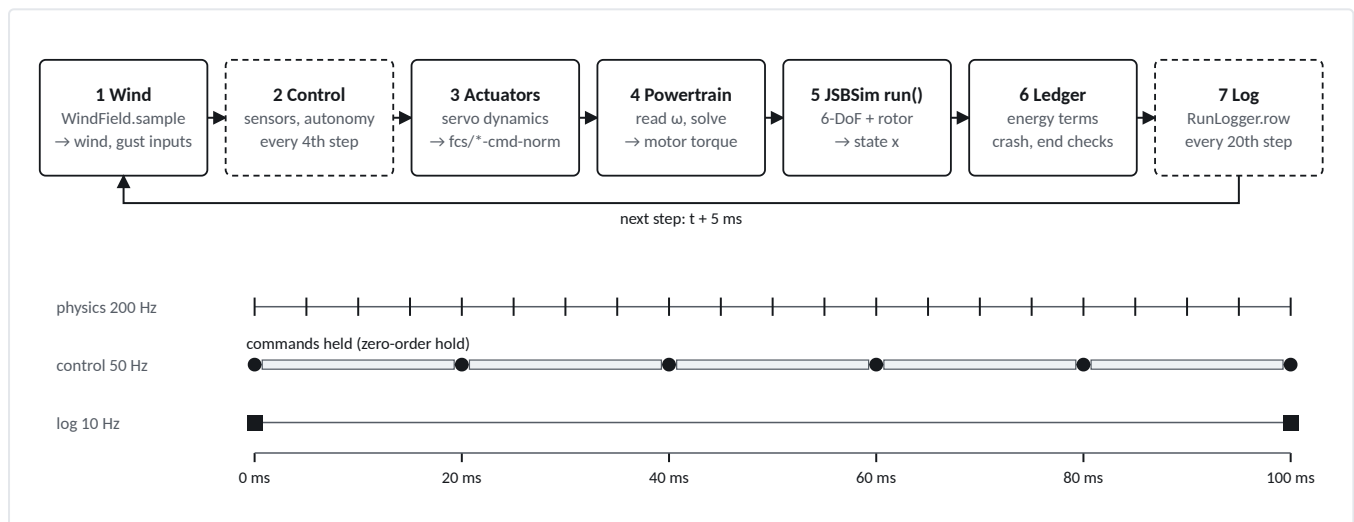


Figure 5. One physics step (top) and the multirate schedule over 100 ms (bottom). Dashed stages run only on their tick. Commands are held between control ticks. Cumulative energies are integrated every step, whatever the log rate.

Rate	What runs
200 Hz (5 ms)	wind, actuators, powertrain, JSBSim, energy ledger, crash check
50 Hz (20 ms)	sensors, autonomy (modes, decisions, policy), guidance, TECS speed/height control, attitude loops
1 Hz	energy-reserve, geofence and energy-feasibility decisions (D4–D6)
10 Hz	one log row

4.3 Determinism and seeds

All randomness (sensor noise, turbulence) comes from one seed. The same configuration and seed give a bit-identical flight on any computer (check V12). That is why a single paired comparison is meaningful here. The optimiser relies on the same property: every candidate design flies with the same seed (common random numbers), so the differences between candidates come only from the design.

5 How the lab applies optimisation

5.1 Four routes into an optimising controller

Route	Method	What it optimises	Where
Offline, from data	parameter sweeps → dataset → surrogate model / Bayesian optimisation	mission settings from <code>dataset_runs.csv</code> and <code>dataset_legs.csv</code>	<code>batch.py</code> , <i>Parameter study</i> page
Online, model-based	speed-to-fly: $\operatorname{argmin}_V P(V)/V_g(V)$ each control step, optional online RLS learning of $P(V)$	airspeed in flight	<code>wind_aware_best_range</code> policy
Design optimisation (this PoC)	cross-entropy method over any parameters, with full flights as evaluations	parameters of a control law, here v_0 and k_{head}	<code>optimize.py</code> , <i>Optimisation</i> page
Learning	reinforcement learning in the Gymnasium environment	a neural policy for airspeed and height	<code>gym_env.py</code>

All four act at the same place, the policy slot of the autonomy layer (Section 2.5). They are evaluated with the same full-flight simulation and the same logs.

5.2 The optimisation problem of this PoC

Decision variables: $x = (v_0, k_{head})$ with bounds $12 \leq v_0 \leq 20$ m/s and $0 \leq k_{head} \leq 1.2$.

Control law flown by the autonomy in every MISSION step:

$$V_{cmd}(t) = \operatorname{clamp} \left(v_0 + k_{head} \hat{H}(t), 12, 26 \right)$$

where $\hat{H}$ is the slow wind estimate resolved against the current leg (positive = head-wind).

Objective: minimise the mission-leg battery energy of a complete simulated flight,

$$J(x) = E_{mission}(x; \text{mission, wind, seed} = 1)$$

Constraints, each violation adding a penalty of 10^6 :

1. the flight ends with status LANDED (no no-go, crash, timeout or failure);
2. the mission is flown completely: the return reason is `mission_complete`.

Why the second constraint matters: a lesson from this PoC. The first optimisation attempt had only constraint 1. In generation 1 it sampled $v_0 = 19.1$ m/s, $k_{head} = 0.94$, which means 26 m/s up-wind. At that speed the battery power is so high that decision D5 predicted only 13 % charge at touchdown, below the 15 % minimum. After 2 min 23 s the autonomy returned and landed safely, exactly as designed. The flight used only 16.9 Wh of mission energy, instead of about 60 Wh, because most of the mission was never flown, and the optimiser ranked it best. **An energy objective must always be paired with a mission-completion constraint.** The interface now sets both constraints by default. In the final run the optimiser sampled the same design again in generation 1, and constraint 2 correctly rejected it.

J has no formula and no gradient. Each evaluation is a 22-minute simulated flight with its turbulence-free but nonlinear dynamics, turns, climbs and autonomy decisions. It is a black-box, simulation-based optimisation problem.

5.3 The cross-entropy method, step by step

CEM keeps a Gaussian search distribution $\mathcal{N}(\mu, \sigma^2)$ over the decision variables. It moves that distribution towards the best designs, generation by generation:

1. **Initialise.** μ is the centre of the bounds, (16 m/s, 0.6). σ is 30 % of the bound range, (2.4 m/s, 0.36).
2. **Sample** a population of $N = 8$ candidates $x_j \sim \mathcal{N}(\mu, \sigma^2)$, clipped to the bounds. Candidate 0 is always μ itself.
3. **Evaluate** every candidate by flying the full mission in the PoC condition with seed 1, in parallel on all CPU cores. Every run is recorded in `evaluations.csv`.
4. **Score** $s_j = J(x_j) + 10^6 \times (\text{number of violated constraints})$. Feasible designs therefore always rank above infeasible ones.
5. **Select** the $N_e = 2$ best candidates (the elite, 25 %).
6. **Update** with smoothing $a = 0.7$ and a floor on σ so that the search never collapses too early:

$$\mu \leftarrow (1 - a) \mu + a \bar{x}_{elite}, \quad \sigma \leftarrow \max \left((1 - a) \sigma + a \text{std}(x_{elite}), 0.02 (hi - lo) \right)$$

7. **Repeat** steps 2–6 for 5 generations. The best feasible design ever seen is the result (`result.json`). The lab then flies it once more with full logs (`best_run/`, and `optimised/` in the PoC folder).

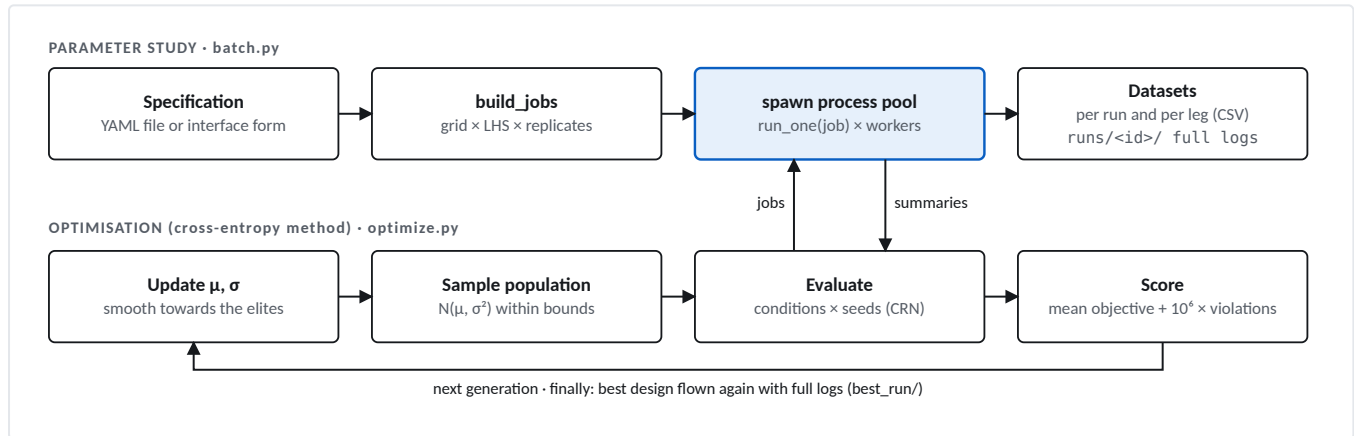


Figure 6. The study engine. A parameter study expands its specification into independent jobs. The optimiser wraps the same runner in a loop: every candidate is flown on every condition with the same seeds (common random numbers), and infeasible designs are pushed down the ranking by a 10^6 penalty.

Why CEM.

- It needs no gradients, so the full nonlinear simulation can be used as it is.
- It tolerates noisy evaluations: the smoothing and elite averaging handle turbulence.
- A whole generation runs in parallel.
- It needs only bounds.

With 2 variables, 8 candidates and 5 generations it costs 40 flights. That is small enough for a laptop and large enough to find the optimum reliably, as the convergence in Figure 7 shows.

Gen.	μ v0 (m/s)	μ k_head	σ v0	σ k_head	best (Wh)	mean feasible (Wh)	feasible
1	15.55	0.646	1.17	0.154	59.01	62.10	7/8
2	15.36	0.657	0.54	0.058	59.03	59.70	8/8
3	15.45	0.658	0.17	0.031	58.81	59.25	8/8
4	15.56	0.660	0.16	0.024	58.82	59.10	8/8

Gen.	μv_0 (m/s)	μk_{head}	σv_0	σk_{head}	best (Wh)	mean feasible (Wh)	feasible
5	15.52	0.668	0.16	0.024	58.83	59.06	8/8

5.4 What the optimiser found

The best design, $v_0 = 15.50$ m/s and $k_{head} = 0.639$, was found in generation 3 with 58.81 Wh. The verification flight with full logs gives 58.82 Wh. The ten best designs lie within 0.20 Wh of each other, so the optimum is well defined in energy but flat in speed. One of the 40 evaluations violated a constraint: $v_0 = 19.1$, $k_{head} = 0.94$. This is the early-return case of Section 5.2, now correctly rejected.

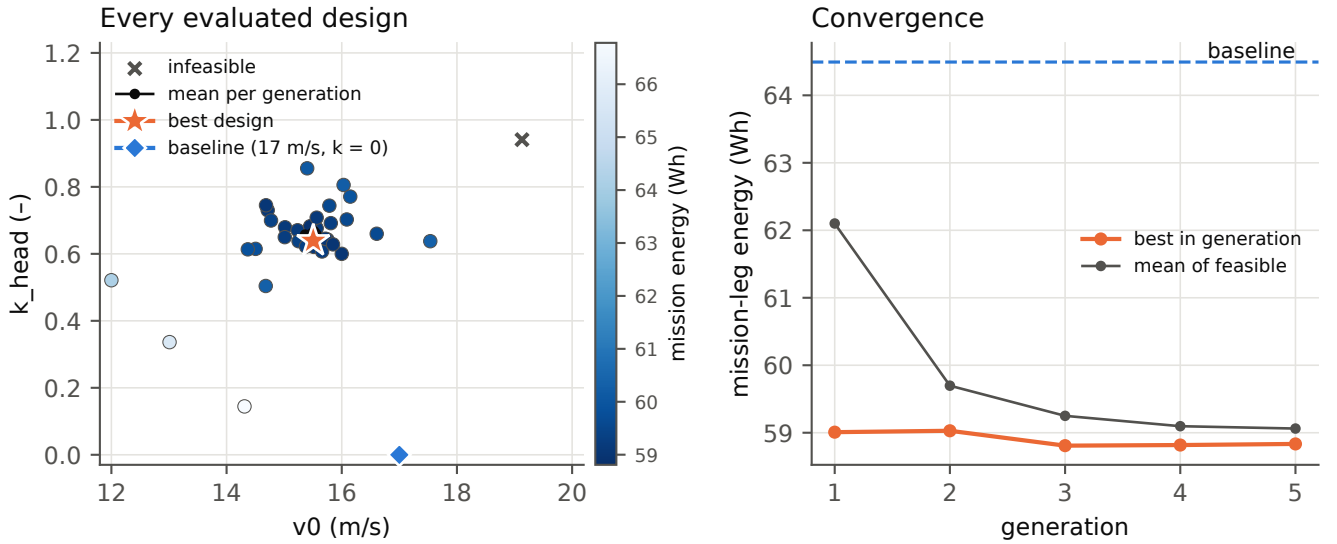


Figure 7. The cross-entropy search. Left: every design evaluated, coloured by mission energy (darker = less energy), with the path of the generation mean and the best design. Right: best and mean feasible mission energy per generation, compared with the baseline.

5.5 The physics behind the result: speed-to-fly

The optimiser is not given any physics. Its result can therefore be checked against classical flight mechanics. The battery power needed in steady level flight, measured with the full pipeline on the **independent** performance map, is fitted as $P(V) = aV^3 + b/V + c$. The ground speed on a leg with head-wind H is $V_g = V - H$. The energy per ground distance is then $P(V)/V_g$, and it is minimised where

$$\frac{d}{dV} \frac{P(V)}{V-H} = 0 \quad \Leftrightarrow \quad P'(V^*) (V^* - H) = P(V^*)$$

With the shipped fit ($a = 0.0203$, $b = 177.8$, $c = 88.5$) and $H = \pm 8.34$ m/s, the theory predicts:

- up-wind: **19.3 m/s**; the optimised law flies 20.8 m/s;
- down-wind: **12.0 m/s** (at the 12 m/s lower limit of the policy); the optimised law flies 12.0 m/s.

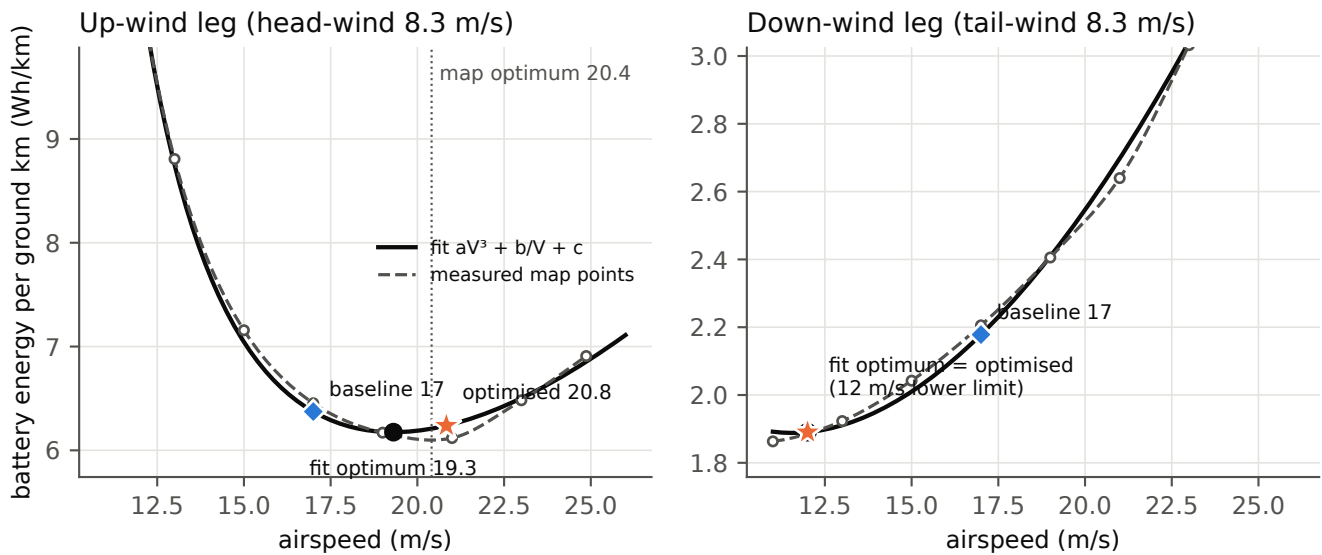


Figure 8. Energy per ground kilometre against airspeed on the up-wind and down-wind legs, from the performance map. Solid: the fitted power curve used by criterion C4, with its optimum (dot). Dashed: the measured map points (post-hoc diagnosis). The fixed 17 m/s (diamond) is too slow into the wind and too fast with it. The optimised law (star) sits at the minimum of the measured curve.

Result of criterion C4: FAIL. The optimiser's up-wind speed is 20.84 m/s, and the fitted formula predicts 19.31 m/s: a difference of 1.53 m/s against the pre-stated tolerance of 1.5 m/s. The down-wind speeds agree: both sit at the 12 m/s lower limit.

Post-hoc diagnosis (added after the result; not a criterion). The full simulation itself prefers about 21 m/s. Designs flying 19.5 m/s up-wind used about 1 Wh more mission energy than the best designs near 20.8–21.1 m/s (Figure 7). The reason is the prediction's power curve. The 3-parameter fit $aV^3 + b/V + c$ overestimates the measured battery power at 21 m/s by 6.1 W (2.2 %), exactly where the optimum lies, and that pulls its optimum down. Repeating the speed-to-fly calculation on the measured performance-map points themselves, with a shape-preserving interpolation instead of the fit, gives an up-wind optimum of **20.4 m/s** (simulation points) and **20.5 m/s** (independent trim-model points). Both are within 0.4 m/s of the optimiser. The energy landscape is also flat here: between 19.3 and 20.8 m/s the up-wind energy per km differs by less than 1 %. For transparency: C4 evaluates the law at the nominal 8.34 m/s head-wind, as implemented before the run. The airspeed actually flown on the up-wind legs averaged 20.1 and 20.7 m/s (Section 7.2), because the in-flight wind estimate and the leg ends differ slightly from the nominal wind.

Consequences. The optimiser, using only complete simulated flights, found the physically correct up-wind speed of the model more accurately than a physics formula built on a smooth fit. The online `wind_aware_best_range` policy uses the same fit and would inherit this bias. It should use the interpolated map instead of the fit. This is a useful finding of the PoC.

5.6 From this PoC to the adaptive optimising controller

- **Robust design.** Optimise over several wind conditions at once: add conditions on the *Optimisation* page. The objective is then the mean over conditions, and the law becomes valid for a range of winds, not just one.
- **More adaptation.** Free k_{cross} , k_{gust} and the altitude offset Δh . Or switch to `wind_aware_best_range` with online RLS, which learns $P(V)$ in flight.
- **Learning.** Train a reinforcement-learning policy in `UAVEnergyEnv` with randomised wind, and compare it with both baselines using exactly this PoC procedure.
- **Hardware.** Calibrate drag, propeller and battery against flight logs (handbook, Section B14.4) before quoting absolute watt-hours. Relative savings such as this PoC's are much less sensitive to calibration.

6 Step-by-step guide to run the proof of concept

Route A: one command (about 21 min on 2 cores, less on more cores)

Step 1: Install once. In PowerShell, in the repository folder, run each line on its own:

```
cd $HOME\Documents\uav_energy_lab
python -m pip install -r requirements.txt
python -m pytest -q
```

Check: the tests end with "24 passed".

Step 2: Run the proof of concept.

```
python poc/run_poc.py
```

In an IDE, open `poc/run_poc.py` and press Run instead. Use `--quick` for a first test (16 optimisation flights) and `--workers N` to set the number of parallel flights.

Check: the console shows STEP 1/6 to STEP 6/6. The last block prints the saving and PASS or FAIL for C1–C6.

Step 3: What each step does and where its output goes (in `poc_results/`):

Step	Action	Output
1	baseline flight, fixed 17 m/s	baseline/ (full logs)
2	CEM optimisation, 8×5 flights	optimisation/ (evaluations.csv, generations.csv, result.json)
3	optimised flight with the best (v_0, k_{head})	optimised/ (full logs)
4	both designs in light turbulence, seeds 11–13	robustness/
5	speed-to-fly prediction from validation_report/performance/performance_fit.json	poc_results.json → physics
6	criteria, tables, figures	poc_results.json, comparison.csv, legs.csv, figures/

Step 4: Read the results. Open `poc_results/figures/`. Or start the interface from the repository folder (`python -m uavlab ui` or `run_ui.py`), go to *Results*, choose the source *Proof-of-concept runs*, tick *baseline* and *optimised*, and press *Compare selected*. The numbers in Section 7 of this document come from `poc_results.json`.

Step 5: If a step failed. Rerun with `--resume`. Finished flights and the finished optimisation are reused, and only the missing steps run.

Route B: in the graphical interface (no code)

1. Start the interface: `python -m uavlab ui`, or run `run_ui.py` in the IDE.
2. **Overview** → template "Head-wind / tail-wind legs" → **Load**. This loads `out_and_back` + `steady_headwind_6`, the PoC condition.
3. **Baseline**: press **Run simulation**. Watch it on **Live run**. When it lands, press **Open results** and label the run "baseline".

4. **Scenario** → **Autonomy & policy**: select the policy `linear_wind`.

5. **Optimisation** page. The defaults already match this PoC:

- variables `autonomy.policy.params.v0` (12–20) and `k_head` (0–1.2);
- objective *Mission-leg battery energy*, *Minimise*;
- constraints `status == LANDED` and `rtl_reason == mission_complete`;
- condition *base*, seed 1, population 8, 5 generations.

Press **Start optimisation** and watch the convergence chart.

6. When it finishes, press **Apply best design to scenario**, then **Run simulation**. Label the run "optimised".

7. **Results** → tick both runs → **Compare runs**. You get the KPI table with differences and the overlaid signals (airspeed, power, energy).

8. (Optional, criterion C5) On **Scenario** → **Wind**, set the turbulence model to *dryden*, the seed to 11, 12 and 13 in turn (*Simulation & logging*), and run both designs for each seed.

Route B uses the same code, settings and optimiser seed, so it gives the same design and the same energies as Route A. This was checked: one candidate flown through both routes gives a bit-identical mission energy.

Troubleshooting

Symptom	Cause and fix
PowerShell: "Missing expression after unary operator '--'"	A multi-line command was pasted. Run one line at a time; every command in this guide is a single line.
"attempted relative import with no known parent package"	A file inside <code>uavlab/</code> was run directly. Run <code>poc/run_poc.py</code> , <code>run_ui.py</code> or <code>python -m uavlab ...</code> instead.
The optimisation is slow	Each flight is about 1 minute of CPU. Use <code>--workers</code> equal to the number of cores, or <code>--quick</code> .
A flight fails with an error	<code>poc_results/optimisation/evaluations.csv</code> shows which candidate. Failed candidates are penalised, not fatal.

7 Results

7.1 Comparison in the PoC condition

Quantity	Baseline (17 m/s)	Optimised law	Difference
Status	LANDED	LANDED	
Mission-leg battery energy (Wh)	64.49	58.82	-8.8 %
Mission-leg energy per km (Wh/km)	4.295	3.963	-7.7 %
Total battery energy, take-off to stop (Wh)	72.94	67.18	-7.9 %
Flight time (min:s)	22:10	19:24	-12.5 %
State of charge at the end (%)	69.8	72.2	+2.4 points
Propeller efficiency, whole flight (%)	76.5	76.4	-0.1 points
Drag work (Wh)	44.18	41.02	-7.1 %
Touchdown sink rate (m/s)	0.64	0.40	
Energy-balance residual (share of gross work)	-3.0e-07	-1.7e-07	

For reference: the physics-based policy. The lab's speed-to-fly policy `wind_aware_best_range` evaluates $\text{argmin}_V P(V)/V_g$ online with the fitted power curve. It flew exactly this condition and seed in V&V check D1 and used **60.33 Wh** (6.5 % less than the baseline). The CEM-tuned law saves 8.8 %. Tuned on complete simulated flights, it does better than the physics policy, which inherits the fit bias described in Section 5.5.

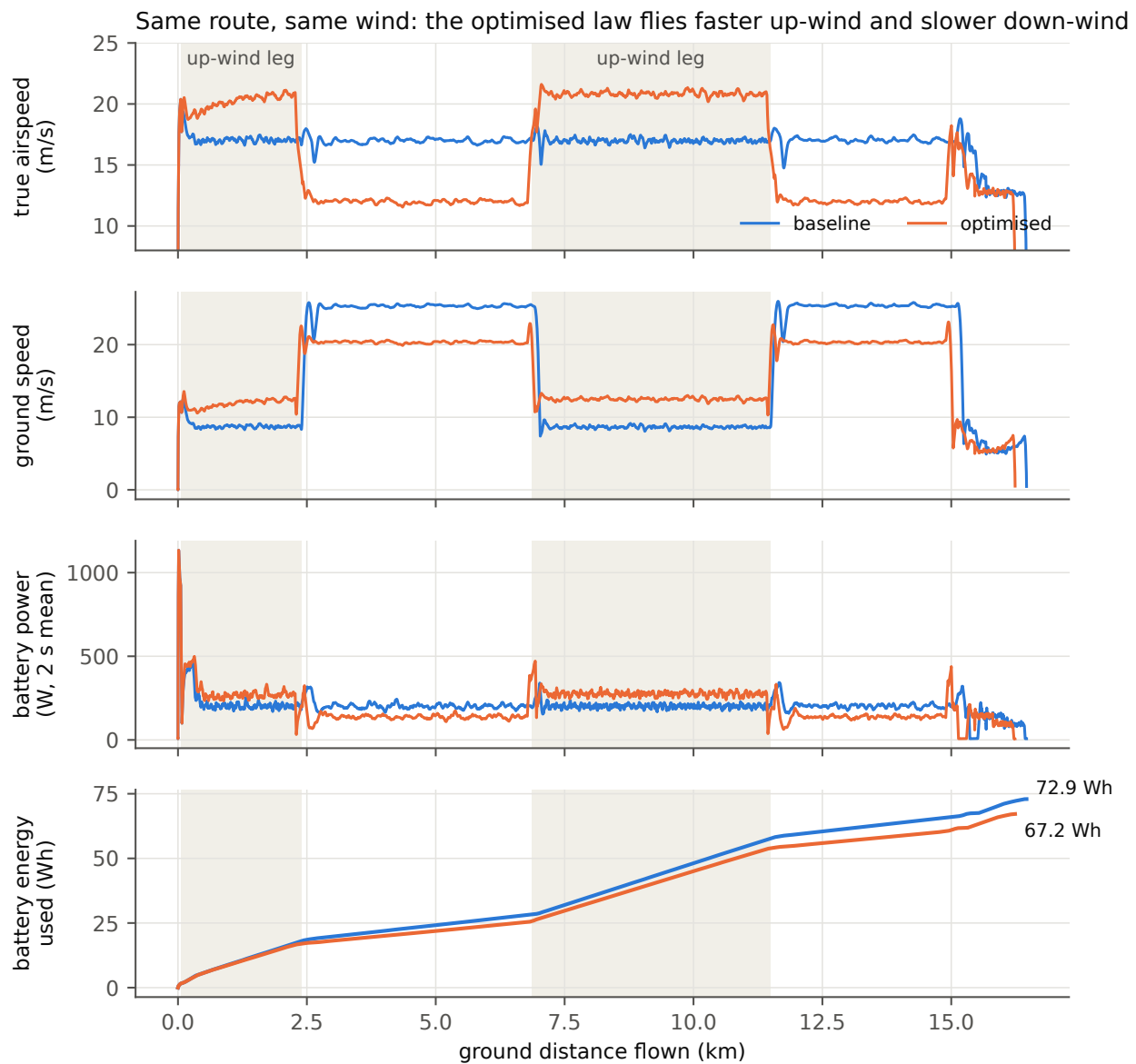


Figure 9. Signals against ground distance, same route and wind. The optimised law flies faster on the shaded up-wind legs (higher ground speed, shorter time against the wind) and slower on the down-wind legs (lower power). The gap in cumulative battery energy opens on every leg.

7.2 Where the saving comes from: leg by leg

Leg	Direction	km	Baseline TAS / GS (m/s)	Baseline Wh/km	Optimised TAS / GS (m/s)	Optimised Wh/km	Change
1 W_END	up-wind	2.33	17.1 / 8.8	7.00	20.1 / 11.9	6.78	-3.2 %
2 E_END	down-wind	4.47	17.0 / 24.8	2.30	12.2 / 20.2	1.91	-17.3 %
3 W_END2	up-wind	4.63	17.0 / 8.8	6.37	20.7 / 12.6	6.13	-3.7 %
4 E_END2	down-wind	3.58	17.0 / 24.7	2.34	12.3 / 20.1	1.93	-17.5 %
all up-wind		6.96	17.0 / 8.8	6.58	20.5 / 12.4	6.34	-3.6 %
all down-wind		8.05	17.0 / 24.7	2.32	12.2 / 20.1	1.92	-17.4 %

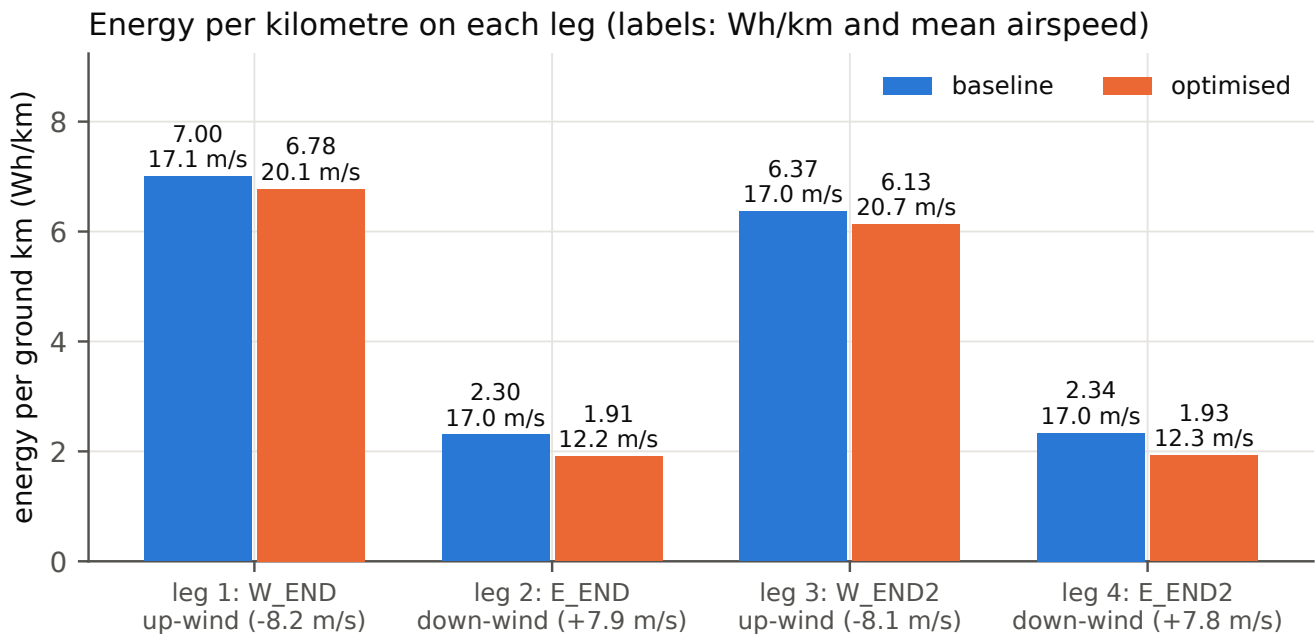


Figure 10. Energy per ground kilometre on each leg. Up-wind legs cost about three times as much per km as down-wind legs. The optimised law saves on both kinds of leg.

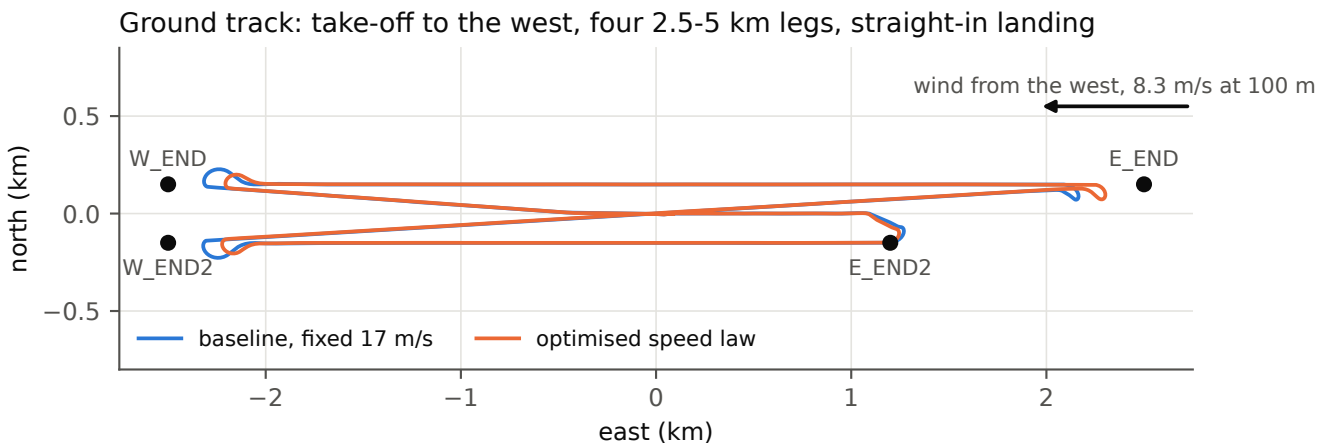


Figure 11. Ground track of both flights: the same route within the guidance tolerance.

7.3 Robustness in unseen turbulence

Seed	Baseline (Wh)	Optimised (Wh)	Change	Status (base / opt)	Return reason (base / opt)
11	66.77	60.36	-9.6 %	LANDED / LANDED	mission_complete / mission_complete
12	64.74	58.58	-9.5 %	LANDED / LANDED	mission_complete / mission_complete
13	65.18	59.05	-9.4 %	LANDED / LANDED	mission_complete / mission_complete

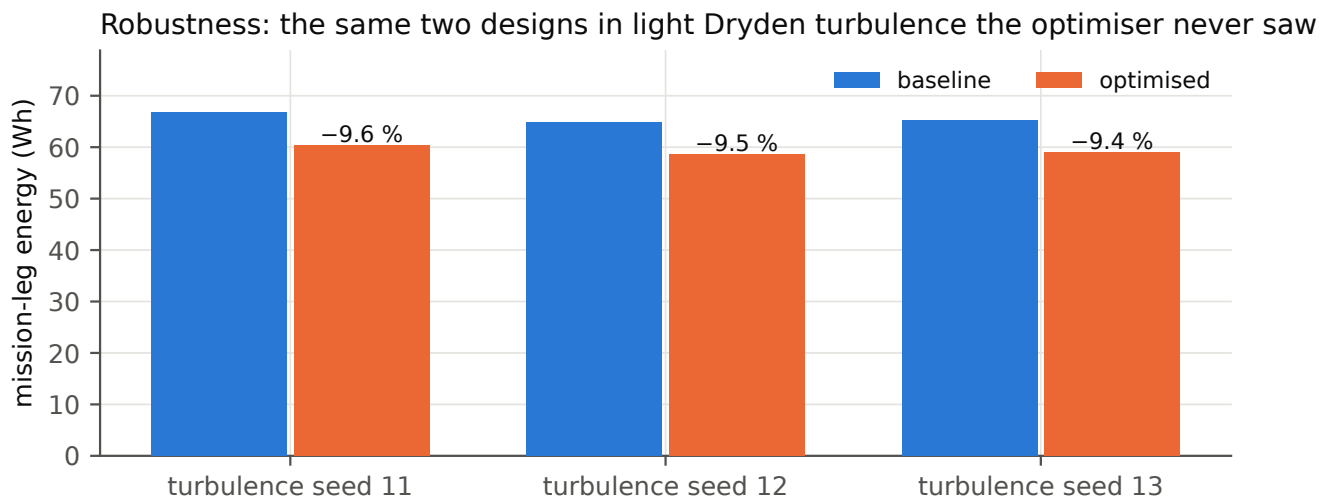


Figure 12. The same two designs in light Dryden turbulence with three seeds the optimiser never saw. Each pair flies identical turbulence.

7.4 Criteria

ID	Criterion	Measured	Verdict
C1	Both flights complete the mission and land on the runway (status LANDED, return reason mission_complete)	baseline LANDED (mission_complete); optimised LANDED (mission_complete)	PASS
C2	Mechanical energy balance closes in both flights: $ \text{residual} < 0.1\%$ of gross work	baseline $-3.0\text{e-}07$; optimised $-1.7\text{e-}07$	PASS
C3	The optimised speed law needs at least 2 % less mission-leg battery energy than the baseline	saving 8.8 %	PASS
C4	Optimised up-wind and down-wind airspeeds agree with the independent speed-to-fly prediction within 1.5 m/s	up-wind 20.84 vs 19.31 m/s (difference 1.53); down-wind 12.0 vs 12.0 m/s. Post-hoc, without the fit: 20.4 m/s (not a criterion)	FAIL
C5	The saving holds in each of 3 unseen light-turbulence seeds (paired comparison, complete missions)	savings 9.6 %, 9.5 %, 9.4 %	PASS
C6	No safety regression: touchdown sink < 2.5 m/s and no energy-reserve decision (D4/D5) in the optimised flight	sink 0.40 m/s; decisions: D1_runway, D2_go	PASS

7.5 Interpretation and limits

Up-wind legs (7.0 km): the optimised law flies 20.5 m/s instead of 17.0 m/s. The ground speed rises from 8.8 to 12.4 m/s, so the time spent against the wind falls from 13:09 to 9:15. The mean battery power is higher (209 → 282 W), but it is needed for less time, and the energy per km falls by 3.6 %.

Down-wind legs (8.1 km): it flies 12.2 m/s instead of 17.0 m/s. The tail-wind still gives 20.1 m/s over the ground, and the battery power drops from 206 to 139 W. The energy per km falls by 17.4 %.

Whole flight. The mission-leg energy falls by 8.8 % and the total by 7.9 %. The flight time changes from 22:10 to 19:24. The physics check (C4) shows why: the fixed 17 m/s is too slow for an 8.3 m/s head-wind and too fast with the same wind behind. The optimiser moved both speeds towards the speed-to-fly optimum.

Limits of this PoC. The absolute watt-hours depend on the representative [REP] aircraft and powertrain data (drag, propeller, battery). The relative saving is much less sensitive to them, but the lab should be calibrated against flight logs before absolute numbers are quoted. The optimised law is tuned to one wind condition. Section 5.6 describes how to extend it to a range of winds. The landing and take-off are identical in both designs, so the saving comes entirely from the mission legs.

Appendix A: Configuration of the experiment

```
# configs/missions/out_and_back.yaml
# Out-and-back 2.5 km legs east-west: isolates head-wind vs tail-wind energy cost per leg.
mission:
  cruise: {airspeed_mps: 17.0, alt_agl_m: 100.0}
  waypoints:
    - {name: W_END, n_m: 150.0, e_m: -2500.0, alt_agl_m: 100.0}
    - {name: E_END, n_m: 150.0, e_m: 2500.0, alt_agl_m: 100.0}
    - {name: W_END2, n_m: -150.0, e_m: -2500.0, alt_agl_m: 100.0}
    - {name: E_END2, n_m: -150.0, e_m: 2500.0, alt_agl_m: 100.0}
  repeat: 1
  geofence: {radius_m: 4000.0, max_alt_agl_m: 150.0}

# configs/wind/steady_headwind_6.yaml
# Steady 6 m/s westerly (at 10 m AGL) with a 1/7 power-law shear profile, no turbulence.
wind:
  mean:
    speed_mps: 6.0
    from_deg: 270.0
    shear: {model: power, alpha: 0.143}
    turbulence: {model: none}

# baseline overrides
autonomy.policy.name: fixed
mission.cruise.airspeed_mps: 17.0

# optimised overrides (result of the optimisation)
autonomy.policy.name: linear_wind
autonomy.policy.params.v0: 15.505
autonomy.policy.params.k_head: 0.6392

# optimisation specification
fixed: {autonomy.policy.name: linear_wind, sim.log_rate_hz: 2}
variables:
  - {path: autonomy.policy.params.v0, lo: 12.0, hi: 20.0}
  - {path: autonomy.policy.params.k_head, lo: 0.0, hi: 1.2}
objective: {kpi: mission_E_Wh, sense: min}
constraints:
  - {kpi: status, op: ==, value: LANDED}
  - {kpi: rtl_reason, op: ==, value: mission_complete}
conditions:
  - label: headwind6
    overrides: {}
seeds: [1]
algorithm: {population: 8, elite_frac: 0.25, iterations: 5, init_sigma_frac: 0.3,
  min_sigma_frac: 0.02, smoothing: 0.7, seed: 0}
```

Appendix B: Files and commands

Item	Location
PoC script	<code>poc/run_poc.py</code> (figures: <code>poc/poc_figures.py</code>)
Results	<code>poc_results/</code> : <code>poc_results.json</code> , <code>comparison.csv</code> , <code>legs.csv</code> , <code>figures/</code> , <code>baseline/</code> , <code>optimised/</code> , <code>optimisation/</code> , <code>robustness/</code>
This document	<code>docs/poc/POC_GUIDE.pdf</code> , source <code>docs/poc/src/poc_guide.md</code> , build <code>tools/poc_guide/</code>
Model and lab reference	<code>docs/handbook/handbook.html</code> , <code>validation_report/VV_REPORT.html</code>

Command	Purpose
<code>python poc/run_poc.py</code>	full proof of concept
<code>python poc/run_poc.py --quick</code>	smaller search for a first check
<code>python poc/run_poc.py --resume</code>	complete an interrupted run
<code>python poc/poc_figures.py</code>	redraw the figures from <code>poc_results/</code>
<code>python -m uavlab ui</code>	graphical interface (Route B)